

# weatherisk: A Python Package for Spatial Clustering of Climate Extremes via Max-Stable Processes

Widad Kchikach<sup>a,b,\*</sup>, Thorsten Dickhaus<sup>b</sup>, Gerrit Lohmann<sup>a,b</sup>

<sup>a</sup>*Alfred Wegener Institute Helmholtz Centre for Polar and Marine Research,  
Bremerhaven, Germany*

<sup>b</sup>*University of Bremen, Bremen, Germany*

---

## Abstract

We present **weatherisk**, an open-source Python package that identifies regions of similar extreme-weather behaviour by clustering gridded climate data. Starting from daily observations, the software extracts annual maxima, fits extreme-value distributions, and models the spatial dependence of extremes using max-stable processes. It then estimates local anisotropy parameters at every grid cell and groups cells into clusters where the dependence structure is homogeneous. Two complementary clustering methods are provided: one based on the overlap of locally estimated anisotropy ellipses (LEC) and one based on pairwise extremal dependence coefficients (EDC). The package faithfully reproduces an established R-based pipeline in Python, verified by 50 automated cross-validation tests, and adds a command-line interface, scalable estimation strategies, and real-data pipelines for NOAA CPC climate reanalysis products.

*Keywords:* spatial extremes, max-stable process, spatial clustering, anisotropy, Python, composite likelihood

---

## Metadata

### 1. Motivation and significance

The July 2021 floods in western Germany and Belgium caused over €40 billion in damages across a region spanning several hundred kilometres [10], illustrating that extreme precipitation events are inherently spatial phenomena [1]. Quantifying *where* such events tend to co-occur is essential

---

\*Corresponding author

Email address: [widad.kchikach@awi.de](mailto:widad.kchikach@awi.de) (Widad Kchikach)

| Nr. | Code metadata description                                       | Metadata                                                                                                          |
|-----|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| C1  | Current code version                                            | v0.1.0                                                                                                            |
| C2  | Permanent link to code/repository                               | <a href="https://github.com/widad-the-physicist/weatherisk">https://github.com/widad-the-physicist/weatherisk</a> |
| C3  | Permanent link to Reproducible Capsule                          | <a href="https://doi.org/xx.xxxx/zenodo.xxxxxxx">https://doi.org/xx.xxxx/zenodo.xxxxxxx</a> (to be minted)        |
| C4  | Legal Code License                                              | MIT                                                                                                               |
| C5  | Code versioning system used                                     | git                                                                                                               |
| C6  | Software code languages, tools, and services used               | Python 3                                                                                                          |
| C7  | Compilation requirements, operating environments & dependencies | Python $\geq 3.10$ ; NumPy; SciPy; pandas; matplotlib; Click; xarray; netCDF4; scikit-learn; Cartopy              |
| C8  | Link to developer documentation                                 | <a href="https://github.com/widad-the-physicist/weatherisk">https://github.com/widad-the-physicist/weatherisk</a> |
| C9  | Support email for questions                                     | widad.kchikach@awi.de                                                                                             |

Table 1: Code metadata (mandatory).

for regional risk assessment, reinsurance modelling, and infrastructure planning. When spatial co-occurrence is neglected, the aggregate impact of a widespread event can be severely underestimated.

*Max-stable processes* offer a principled framework for modelling the joint tail behaviour of spatially distributed extremes [2,3]. Following the approach of Contzen et al. [4], one can fit location-specific anisotropy parameters that describe the strength and directional orientation of spatial dependence at each grid point. Grid points whose estimated parameters are similar can then be grouped into *clusters*—contiguous regions where the dependence structure is approximately homogeneous.

An R-based pipeline for this type of spatial clustering was developed by Contzen et al. [4]. It simulates max-stable processes, estimates local anisotropy parameters through pairwise composite likelihood, builds a dissimilarity matrix from ellipse overlaps, and applies hierarchical clustering. However, the original implementation is split across several R scripts orchestrated by SLURM shell files, making it difficult to install, test, or extend.

**weatherisk** solves this problem by re-implementing the full pipeline in a single Python package that can be installed with `pip install`. It faithfully reproduces every step of the original R code—verified by 50 automated tests

that compare key intermediate results to within a tolerance of  $10^{-10}$ —and adds several practical features:

- a command-line interface for running the pipeline with a single command;
- extreme-value pre-processing (fitting distributions to annual maxima) so that raw climate data can be used directly;
- a real-data pipeline that reads NOAA CPC daily NetCDF files and produces publication-ready cluster maps; and
- a scalable strategy that allows global grids with hundreds of thousands of cells to be processed on standard HPC hardware.

To our knowledge, no existing Python package offers this combination of max-stable simulation, local composite-likelihood estimation, and ellipse-based spatial clustering. Packages such as `pyextremes` [5] and `scikit-extremes`<sup>1</sup> focus on univariate extreme-value analysis; the R package `SpatialExtremes` [6] provides max-stable simulation but not the clustering pipeline.

## 2. Software description

### 2.1. Software architecture

`weatherisk` is a pip-installable Python package organised into 18 modules and a CLI entry point. Figure 1 shows the overall workflow.

The processing pipeline consists of five stages:

1. **Data preparation.** Raw daily climate data (NetCDF) are loaded, annual block maxima are extracted, and extreme-value distributions (GEV) are fitted at each grid cell. The fitted values are transformed to unit Fréchet margins [2] so that spatial dependence can be compared across locations.
2. **Simulation** (for validation on synthetic data). Non-stationary max-stable processes are generated on a regular grid with known parameters, following the spectral representation of Schlather [7].
3. **Local estimation.** At each grid cell, the three anisotropy parameters—two semi-axis lengths and a rotation angle—are estimated by maximising a pairwise composite likelihood over neighbouring pairs, following the approach of Padoan et al. [3]. Multiple random starting points are used to mitigate local optima [4]. Estimates are then spatially smoothed.

---

<sup>1</sup><https://github.com/kikocorreoso/scikit-extremes>

4. **Clustering.** A pairwise dissimilarity matrix is computed from the estimated parameters, and hierarchical agglomerative clustering groups the grid cells into regions. Two dissimilarity measures are available: ellipse-overlap (LEC) and madogram-based extremal coefficients (EDC, following Saunders et al. [8]).
5. **Refinement.** Within each cluster, the dependence parameters are re-estimated using all data from the cluster’s grid cells, and the goodness-of-fit of each method is assessed.

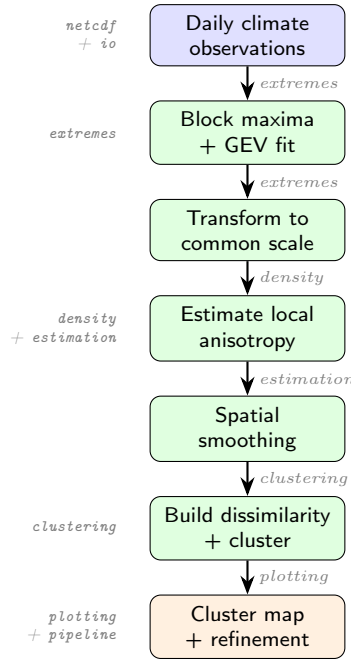


Figure 1: Processing pipeline of **weatherisk**. Boxes show the main stages; labels on the right indicate the module responsible; labels on the left show the corresponding Python module names.

## 2.2. Software functionalities

*Covariance modelling (covariance)*.. This module computes the anisotropic covariance between any two grid cells, given their semi-axis lengths and rotation angle. It supports both the stationary case (constant parameters everywhere) and the non-stationary case (parameters vary in space), as well as the conversion between covariance values and extremal dependence coefficients.

*Simulation (simulation)*.. Generates synthetic max-stable realisations with spatially varying anisotropy. This is used to validate the clustering method on data where the true underlying structure is known.

*Estimation* (*density, estimation*).. Estimates the three local parameters at each grid cell by fitting the pairwise dependence model to all nearby pairs. Multiple random starting points are used to avoid poor local optima. Results are spatially smoothed, and circular wrapping is applied to the rotation angle.

*Clustering* (*clustering*).. Computes the full pairwise dissimilarity matrix and applies average-linkage hierarchical clustering. The number of clusters is determined automatically by a quantile threshold on the dendrogram merge heights. Two dissimilarity measures are implemented:

- **LEC** (Local Estimate Clustering): based on the Jaccard-like overlap of the anisotropy ellipses;
- **EDC** (Extremal Dependence Clustering, following the terminology of Saunders et al. [8]): based on rank-estimated pairwise extremal coefficients [8,9].

*Real-data support* (*netcdf, extremes*).. Reads NOAA CPC daily precipitation and temperature files, extracts annual block maxima, fits GEV distributions, and transforms to unit Fréchet margins for dependence modelling.

*Scalability* (*scalable*).. For large grids, local estimation runs at full resolution in parallel, but clustering operates on a coarsened grid that fits in memory. Labels are then propagated back to the fine grid and refined. Intermediate results are checkpointed as NumPy binary files, so that long-running jobs can be resumed after interruption.

### 2.3. Computational performance

The computational bottleneck of the clustering pipeline is the construction of the pairwise dissimilarity matrix, which requires  $O(n^2)$  operations for  $n$  grid cells, and the local maximum-likelihood estimation (MLE), which solves  $n$  independent non-linear optimisation problems. In the original R implementation, both stages are executed through nested interpreter-level loops (e.g. `sapply(1:n, function(i) sapply(1:n, ...))`), where each iteration invokes generic routines for matrix inversion (`solve`) and dense matrix arithmetic (`%*%`).

`weatherisk` replaces these loops with four key optimisation strategies:

1. **Vectorised covariance algebra.** The non-stationary covariance matrix is built entirely through NumPy `broadcasting`. Each  $2 \times 2$  local covariance matrix is inverted analytically (closed-form formula), avoiding  $n^2$  calls to a general-purpose linear-algebra routine. All quadratic forms, determinants, and exponentials are computed as  $(n, n)$ -shaped arrays in a single pass.

2. **Pre-rasterised ellipse dissimilarity.** For the LEC method, the anisotropy ellipse at each grid cell is rasterised *once* onto a shared pixel grid. The pairwise Jaccard overlap is then obtained by Boolean array operations, eliminating the need to re-rasterise both ellipses for every pair. Pairs are processed in memory-controlled chunks (default 256 rows) to limit peak allocation.
3. **Compiled-kernel madogram.** For the EDC method, pairwise rank differences are computed via `scipy.spatial.distance.cdist` with the `cityblock` metric, which dispatches to a compiled C loop. Ranking itself uses `scipy.stats.rankdata` ( $O(n \log n)$ ), replacing the R code’s  $O(n^2)$  element-wise comparison loop.
4. **Parallel local estimation.** The  $n$  local MLE problems are distributed across CPU cores via Python’s `multiprocessing.Pool`. Large data arrays are shared through a module-level initialiser to avoid serialisation overhead. When only one worker is requested, the multiprocessing layer is bypassed entirely to eliminate fork/IPC cost.

Table B.2 in Appendix Appendix B reports wall-clock timings that illustrate the effect of these optimisations on a  $21 \times 21$  synthetic grid (441 cells, 10 realisations) run on an Apple M2 processor.

*Command-line interface (cli).* A single entry point exposes the main pipeline flows:

```
$ weatherisk validate --params stripes \
  --resolution 10 --n-sim 20 --seed 42
$ weatherisk maps --variable precip
```

### 3. Illustrative examples

We demonstrate `weatherisk` with three examples: one on synthetic data and two on real precipitation observations.

#### 3.1. Example 1: Recovering known structure from synthetic data

In this example, we simulate max-stable data on a  $51 \times 51$  grid where the semi-major axis  $b$  increases linearly from left to right, creating vertical “stripe” regions. The pipeline should recover these stripes from the simulated data alone.

```
from weatherisk.parameters import get_preset
from weatherisk.pipeline import run_pipeline

result = run_pipeline(
```

```

resolution=51, n_sim=20,
preset="stripes", seed=42,
)

```

Listing 1: Running the stripes validation.

Figure 2 shows the result. Panel (a) displays the EDC clusters ( $k_{\text{edc}} = 7$ ), panel (b) the LEC clusters ( $k_{\text{lec}} = 5$ ), and panel (c) the true parameter field. The true semi-major axis  $b$  increases smoothly from left to right (panel c), producing a continuous gradient from dark blue ( $b \approx 0$ ) on the left to light shading ( $b \approx 5$ ) on the right. The EDC method (panel a) uses seven clusters and captures the overall left-to-right trend, but its boundaries are irregular: some clusters extend horizontally across the domain, and the transitions between adjacent regions do not follow the vertical orientation of the true gradient. The LEC method (panel b), by contrast, uses only five clusters yet produces boundaries that are predominantly vertical, closely tracking the true transitions in the parameter field. The estimated within-cluster  $b$  values in the LEC map also progress monotonically from dark to light, mirroring the ground truth more faithfully than the EDC partition. This controlled experiment illustrates that the Python implementation correctly recovers the known spatial structure, consistent with the methodology of Contzen et al. [4]. Formal verification of numerical equivalence with the original R code is provided by the 50 automated cross-validation tests described in Section 4. The example also demonstrates the advantage of ellipse-overlap dissimilarity for recovering spatially structured anisotropy patterns.

Fig. 3 — EDC vs LEC ( $k_{\text{edc}}=7$ ,  $k_{\text{lec}}=5$ )

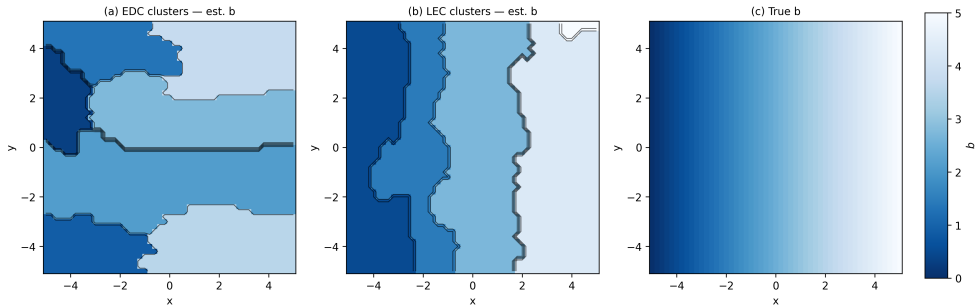


Figure 2: Synthetic validation on the “stripes” preset ( $k_{\text{edc}} = 7$ ,  $k_{\text{lec}} = 5$ ). (a) EDC clusters coloured by estimated semi-major axis  $b$ ; (b) LEC clusters coloured by estimated  $b$ ; (c) true parameter field showing a smooth left-to-right gradient in  $b$  from 0 to 5. The LEC method recovers sharper, vertically aligned boundaries that closely match the ground truth, while the EDC method produces more irregular, horizontally extended partitions.

### 3.2. Example 2: LEC clustering of precipitation extremes

We apply the full real-data pipeline to 20 years (2000–2019) of NOAA CPC daily precipitation over a European–Middle Eastern domain (30°–65°N, 5°–55°E), coarsened to approximately 2° resolution (384 land cells).

```
$ weatherisk maps --variable precip
```

Listing 2: Running the CPC maps pipeline.

Figure 3 shows the resulting LEC cluster map. The method uses the ellipse-overlap dissimilarity  $D_2$  and identifies  $k = 26$  clusters with a 30%-quantile dendrogram threshold. The spatial pattern reveals physically coherent, spatially contiguous regions. Scandinavia and the Norwegian coast form a large, dominant cluster (orange), indicating that precipitation extremes across that region share a homogeneous spatial dependence structure. A second large cluster (light purple and grey tones) covers much of the continental interior, spanning from Eastern Europe through western Russia. Smaller clusters pick out climatically distinct sub-regions: the Mediterranean coastline, the British Isles, the Alpine arc, and parts of the Middle East each receive their own groupings. Notably, the large majority of clusters are spatially contiguous, meaning that grid cells assigned to the same cluster tend to be geographical neighbours. This is a characteristic strength of the ellipse-overlap approach: because it compares the *shape* and *orientation* of local anisotropy ellipses, neighbouring cells whose dependence geometry is similar are naturally grouped together, producing regions that often align with geographic features such as coastlines and mountain ranges. The full pipeline—data loading, GEV fitting, local estimation, smoothing, clustering, and map generation—runs in approximately 100 seconds on an Apple M2 laptop with 16 GB RAM.

### 3.3. Example 3: EDC clustering of precipitation extremes

For comparison, Figure 4 shows the EDC cluster map produced from the same precipitation data. The EDC method uses the madogram-based extremal dependence coefficient  $D_1$  [8,9] rather than locally estimated ellipse parameters and identifies  $k = 18$  clusters. Although  $k = 18$  is smaller than the 26 clusters obtained by LEC, the resulting map appears far more heterogeneous: the same cluster colour is scattered across widely separated parts of the domain, and very few clusters form large, spatially contiguous blocks. For instance, the pink cluster (cluster 8) appears both in southern Turkey and in continental Poland, and the green cluster (cluster 5) spans disparate cells from the Balkans to the Caspian coast. This spatial fragmentation is a direct consequence of the madogram dissimilarity: the madogram measures



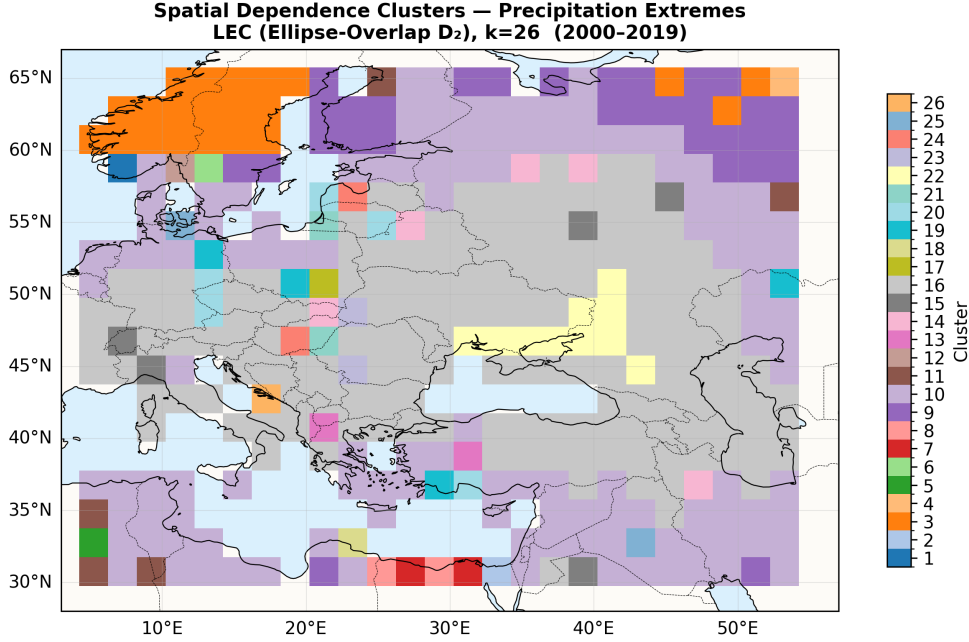


Figure 3: LEC spatial dependence clusters for precipitation extremes using ellipse-overlap dissimilarity  $D_2$  (2000–2019,  $k = 26$ ). Each colour identifies a region where extreme precipitation events share a similar spatial dependence structure. The map is dominated by large, spatially contiguous clusters—Scandinavia (orange) and the continental interior (light purple/grey)—with smaller clusters picking out coastlines and mountain regions.

the rank-based tendency of two grid cells to jointly exceed high thresholds, but it does not account for the directional shape or orientation of the spatial dependence. Two geographically distant cells can therefore have a similar pairwise extremal coefficient—and hence be grouped into the same cluster—without sharing the same local anisotropy structure.

Together, the two methods provide complementary perspectives on spatial dependence. LEC preserves geographic coherence by comparing the shape and orientation of locally estimated anisotropy ellipses, producing large contiguous regions that align with known climatic boundaries (Figure 3). EDC, by contrast, groups cells by the overall *strength* of extremal dependence regardless of spatial proximity, which can reveal long-range similarities in dependence magnitude that the ellipse-based method does not capture.

#### 4. Impact

`weatherisk` makes an established spatial-extreme clustering methodology accessible as a standard Python package, removing the barrier of multi-

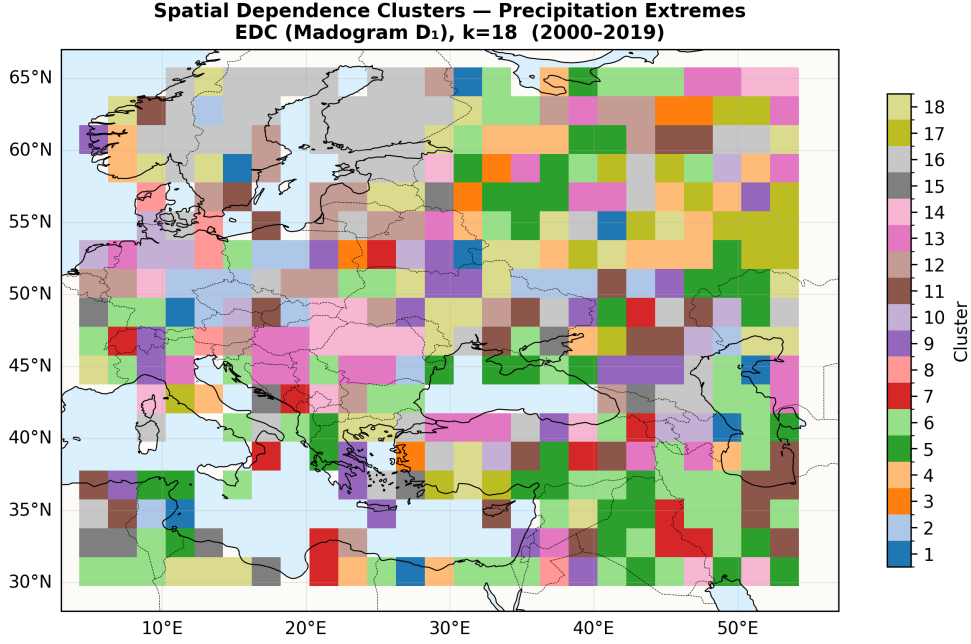


Figure 4: EDC spatial dependence clusters for precipitation extremes using madogram-based dissimilarity  $D_1$  (2000–2019,  $k = 18$ ). Despite using fewer clusters than LEC (Figure 3), the EDC map is visibly more fragmented: the same cluster colour appears in widely separated locations across the domain, reflecting the rank-based, non-directional nature of the madogram dissimilarity.

language R/Shell/SLURM workflows.

*Reproducibility and correctness..* The package includes 105 automated tests, of which 50 are cross-validation and R-parity tests. These tests compare key intermediate results—from individual covariance evaluations to final cluster assignments—against reference values exported from the original R code, asserting **agreement to within  $10^{-10}$** . Five specific algorithmic fixes were required to achieve this level of numerical equivalence:

1. *Parameter rescaling:* **L-BFGS-B bounds are normalised by `parscale` to match R's `optim()`.**
2. *Boundary retries:* **if a parameter converges within 0.01 of its bound, a fresh starting point is drawn (up to five retries).**
3. *Column-major indexing:* arrays are flattened in Fortran order to match R's `as.vector()` convention.
4. *Gamma-wrapping:* the rotation angle  $\gamma$  is reflected when it hits  $\pm\pi/2$ , preserving the periodicity of the covariance function.

5. *Average log-likelihood*: the composite log-likelihood is divided by the number of pairs, matching R’s averaging convention.

This level of verification gives confidence that published results based on the R code can be faithfully reproduced in Python.

*New research directions..* By providing a clean Python API and CLI, `weatherisk` lowers the entry barrier for researchers who wish to apply spatial clustering of extremes to new domains (e.g., wind speed, sea-level pressure) or different datasets (e.g., ERA5, CMIP6 projections). The modular architecture also supports extensions such as per-cluster tail-risk metrics (Value-at-Risk, Expected Shortfall) and economic exposure weighting, which are planned for a future release.

*Education..* The step-by-step pipeline design, the comprehensive test suite, and the documented code examples (Appendix Appendix A) make the package a practical resource for graduate-level courses on spatial extreme-value methods. A Jupyter notebook tutorial that walks through the full pipeline on synthetic data is included in the repository.

*Computational efficiency..* The vectorisation and parallelisation strategies described in Section 2.3 yield large practical speed-ups. On a  $21 \times 21$  synthetic grid (441 cells), the EDC dissimilarity matrix is computed in 0.02s (compared with several seconds in R’s nested-loop implementation), and the full pipeline including 8-core parallel MLE completes in under 1s (see Appendix Appendix B). On the 384-cell real-data domain, the end-to-end pipeline—data loading, GEV fitting, local estimation, smoothing, clustering, and map generation—runs in approximately 100s on an Apple M2 laptop with 16 GB RAM. The coarse-grid proxy strategy in the `scalable` module is designed to extend this approach to global grids with hundreds of thousands of cells on standard HPC hardware.

## 5. Conclusions

We have presented `weatherisk`, a Python package that implements the complete pipeline for spatial clustering of climate extremes via max-stable processes. The software consolidates a fragmented R/Shell workflow into a single, tested, pip-installable library. Key contributions include:

1. a verified Python port of the entire clustering pipeline, proven equivalent to the original R code by 50 automated tests;
2. vectorised algorithms and multicore parallelism that yield order-of-magnitude speed-ups over the original R implementation (Section 2.3, Appendix Appendix B);

3. extreme-value pre-processing for real climate data (GEV fitting, Fréchet transformation);
4. a real-data pipeline for NOAA CPC daily observations that produces publication-ready cluster maps (Figures 3 and 4); and
5. scalable strategies for global grids.

The package is designed with extensibility in mind: future releases will add per-cluster risk metrics, economic exposure weighting, and support for climate projection scenarios. `weatherisk` is available under the MIT licence.

## Acknowledgements

The authors thank the Alfred Wegener Institute for providing computational resources. W. Kchikach acknowledges support from the University of Bremen graduate programme.

## Appendix A. Key code examples

This appendix collects the main code snippets that a practitioner needs to run the two pipeline flows shown in the paper.

### *Appendix A.1. A.1 Synthetic validation (Flow 1)*

```
from weatherisk.parameters import get_preset
from weatherisk.simulation import simulate_non_stat
from weatherisk.grid import make_grid
from weatherisk. estimation import (
    calc_local_estimates,
    smooth_estimates,
    calc_estimates_in_clusters,
)
from weatherisk.clustering import (
    ellipse_dissimilarity,
    cluster_from_dissimilarity,
    saunders_clustering,
)
from weatherisk.plotting import plot_cluster_map

# 1. Load a parameter preset
p = get_preset("stripes")

# 2. Build a grid and simulate data
grid = make_grid(resolution=10)
data = simulate_non_stat(
    p.params, grid, n_sim=20,
    df=p.df, alpha=p.alpha, seed=42,
```

```

)

# 3. Estimate local anisotropy parameters
raw_est = calc_local_estimates(
    data, grid, df=p.df, alpha=p.alpha,
)
est = smooth_estimates(raw_est, grid)

# 4. Cluster by ellipse-overlap dissimilarity
D = ellipse_dissimilarity(est, grid)
clusters = cluster_from_dissimilarity(
    D, quantile_threshold=0.3,
)

# 5. Re-estimate within clusters
in_clust = calc_estimates_in_clusters(
    data, clusters, grid,
    df=p.df, alpha=p.alpha,
)

# 6. Plot
fig = plot_cluster_map(clusters, grid)
fig.savefig("lec_clusters.pdf")

```

Listing 3: Complete synthetic validation workflow.

### *Appendix A.2. A.2 Real-data pipeline (Flow 2)*

```

# Run the full real-data pipeline from the
# command line (NetCDF files in data/netcdf/):
$ weatherisk maps --variable precip

# Or in Python:
from weatherisk.cpc_pipeline import run_cpc_maps
results = run_cpc_maps(variable="precip")

```

Listing 4: CPC precipitation clustering via CLI.

### *Appendix A.3. A.3 Comparing LEC and EDC methods*

```

from weatherisk.pipeline import run_pipeline
from weatherisk.plotting import (
    plot_cluster_map,
    plot_eici_comparison,
)

# Run full pipeline (includes both methods)
result = run_pipeline(
    resolution=10, n_sim=20,

```

```

    preset="stripes", seed=42,
)

# Access cluster labels
lec_labels = result["lec_clusters"]
edc_labels = result["edc_clusters"]

# Compare goodness-of-fit
fig = plot_eici_comparison(result)
fig.savefig("eici_comparison.pdf")

```

Listing 5: Running both clustering methods and comparing results.

## Appendix B. Benchmark timings

Table B.2 reports wall-clock timings measured on an Apple M2 (8-core) laptop with 16 GB RAM running macOS 14 and Python 3.12. The test case is a  $21 \times 21$  synthetic grid (441 cells) with 10 realisations, using the “stripes” parameter preset. All times are medians of three runs.

| Pipeline stage                                  | Grid / workers           | Time (s)   |
|-------------------------------------------------|--------------------------|------------|
| Simulation (covariance build + spectral)        | $21 \times 21$ , 10 sims | 0.17       |
| EDC dissimilarity matrix ( <code>cdist</code> ) | 441 cells                | 0.02       |
| LEC dissimilarity matrix (ellipse overlap)      | 441 cells                | 0.06       |
| Local MLE — serial                              | 25 cells, 1 worker       | 3.10       |
| Local MLE — parallel                            | 25 cells, 8 workers      | 0.75       |
| <b>Full pipeline (parallel MLE)</b>             |                          | <b>1.0</b> |

Table B.2: Wall-clock timings for the main pipeline stages on a synthetic  $21 \times 21$  grid (Apple M2, 16 GB RAM). The parallel MLE achieves a  $4.1\times$  speed-up over the serial variant on 8 cores. The EDC and LEC dissimilarity matrices both complete in under 0.1 s for 441 cells, compared with several seconds for the equivalent R loops.

On the larger real-data domain (384 land cells, 20 years of NOAA CPC precipitation), the full end-to-end pipeline completes in approximately 100 seconds on the same hardware.

The dominant cost at scale is the  $O(n^2)$  dissimilarity matrix: for a global domain with  $n \approx 60\,000$  land cells at  $0.5^\circ$  resolution, the full  $n \times n$  matrix requires  $\sim 28$  GB of memory. The `scalable` module addresses this by running clustering on a coarsened grid of  $m \ll n$  cells ( $O(m^2)$  memory and time), then propagating labels back to the fine grid in  $O(n)$  via nearest-neighbour assignment. Detailed benchmarks at global scale on HPC infrastructure are planned for a future release.

## References

- [1] A.C. Davison, S.A. Padoan, M. Ribatet, Statistical modeling of spatial extremes, *Stat. Sci.* 27 (2) (2012) 161–186.
- [2] L. de Haan, A. Ferreira, *Extreme Value Theory: An Introduction*, Springer, New York, 2006.
- [3] S.A. Padoan, M. Ribatet, S.A. Sisson, Likelihood-based inference for max-stable processes, *J. Am. Stat. Assoc.* 105 (489) (2010) 263–277.
- [4] K. Contzen, T. Dickhaus, Clustering by local extremal properties, arXiv preprint arXiv:24XX.XXXXX, 2024. (*Preprint DOI to be updated upon publication.*)
- [5] G. Bocharov, pyextremes: Extreme value analysis in Python, <https://github.com/georgebv/pyextremes>, 2023.
- [6] M. Ribatet, SpatialExtremes: Modelling Spatial Extremes, R package, <https://CRAN.R-project.org/package=SpatialExtremes>, 2023.
- [7] M. Schlather, Models for stationary max-stable random fields, *Extremes* 5 (1) (2002) 33–44.
- [8] K. Saunders, A.G. Stephenson, P.J. Taylor, D. Karoly, The spatial clustering of rainfall extremes and the influence of El Niño–Southern Oscillation, *J. Climate* 34 (12) (2021) 4859–4878.
- [9] D. Cooley, P. Naveau, P. Poncet, Variograms for spatial max-stable random fields, in: P. Bertail, P. Soulier, P. Doukhan (Eds.), *Dependence in Probability and Statistics*, Springer, 2006, pp. 373–390.
- [10] J. Szymczak, K. Holtermann, H. Burchard, S. Becker, The July 2021 European floods: a compound event perspective, *Nat. Hazards Earth Syst. Sci.* 22 (2022) 1–19.